

Key-Recovery Side-Channel Attack on the Berlekamp-Massey Decoding Algorithm in the *Classic McEliece* KEM

Andrei Alexei¹, Marios Omar Choudary¹ and Vlad-Florin Drăgoi²

¹ Computer Science Department, Faculty of Automatic Control and Computer Science,
National University of Science and Technology POLITEHNICA Bucharest,
060042 Bucharest, Romania,

andrei.alexei@stud.acs.upb.ro, marios.choudary@upb.ro

² Faculty of Exact Sciences, Aurel Vlaicu University of Arad,
Arad, Romania,

vlad.dragoi@uav.ro

Abstract. In this article, we provide the first side-channel attack on the Berlekamp-Massey (BM) algorithm, which is the decoder used in the decryption process of the *Classic McEliece* KEM. We conduct a chosen plaintext key recovery attack that exploits the power consumption of the BM, which is highly dependent on the secret Goppa support elements. We exploit the relation between plaintexts of small Hamming weight, secret elements in the Goppa support and power traces using an efficient Template Attack. Our method completely recovers the secret Goppa support for the first parameter set of the *Classic McEliece* KEM using a single attack trace per secret coefficient. The entire support can be recovered in less than 7 seconds on a standard computer. Our experiments are performed using the ChipWhisperer-Lite board platform with the ARM Cortex-M4 microcontroller.

Keywords: No keywords given.

1 Introduction

The advent of quantum computing poses a significant threat to classical public-key cryptographic systems. Several quantum algorithms enable efficient optimizations for the factorization of large integers and the computation of discrete logarithms on a quantum computer, thus compromising public-key encryption schemes such as RSA and ECC [Gro96, Sho94]. Although large-scale quantum computers capable of executing Shor’s algorithm [Sho94] are not yet a reality, rapid advances in quantum technologies suggest their feasibility in the near future, e.g., the new record of 6100 highly reliable qubits [MNB⁺25]. Such advances spurred the development of post-quantum cryptography (PQC). Despite being quite a new process, post-quantum encryption schemes exist since 1978. Indeed, Robert J. McEliece proposed the well-known McEliece cryptosystem, the first code-based cryptosystem [McE78], whose security relies on the hardness of decoding linear codes, an NP-complete problem [BMvT78].

1.1 Standardization of post-quantum cryptography

In response to the quantum threat, the National Institute of Standards and Technology (NIST) initiated a Post-Quantum Cryptography Standardization process in 2016. The set

of finalists was announced in two rounds (2024 and 2025): ML-KEM (based on CRYSTALS-Kyber) [oST24a], ML-DSA (based on CRYSTALS-Dilithium) [oST24b], SLH-DSA (based on SPHINCS+) [oST24c], FN-DSA (based on Falcon) [PFH⁺22], and HQC [AAB⁺22]. In its last PQC standardization round, the NIST stated that other strong candidates, such as *Classic McEliece* [ABC⁺22] were left for standardization with other organizations. Concretely, besides NIST’s efforts, ISO and the International Electrotechnical Commission (IEC) have released guidance on migration towards PQ-cryptography. They have already standardized stateful hash-based signatures (e.g., LMS, XMSS) in ISO/IEC 14888-4 and are working on a PQC update for ISO/IEC 18033-2, to incorporate *Classic McEliece* and FrodoKEM – two conservative, well-studied key encapsulation mechanisms. *Classic McEliece*, in particular, is valued for its resilience against quantum attacks and its long-standing security record, making it a strong candidate for future industrial adoption.

In its Technical Guideline BSI TR-02102-1 [fSidI25], the BSI has assessed FrodoKEM (specifically FrodoKEM-976 and FrodoKEM-1344) and *Classic McEliece* (with parameter sets mceliece460896, mceliece6688128, and mceliece8192128) as cryptographically suitable for protecting confidential information on a long-term basis. This recommendation is based on a conservative assessment that includes a significant margin of security against future cryptanalytic advances. In particular, *Classic McEliece* is considered *very thoroughly analysed* (see [fSidI25, Section 2.4.2]).

1.2 High-level description of the *Classic McEliece* KEM

Albrecht et al. introduced the *Classic McEliece* algorithm [ABC⁺22], as a classical and robust code-based KEM construction. The key generation is a well-known masking technique of a structured linear code, in this case a binary irreducible Goppa code. The private key is the pair $(g(x), \mathcal{L})$, an irreducible Goppa polynomial $g(x) \in \mathbb{F}_{2^m}[x]$ of degree t and a private Goppa support, which is a subset $\mathcal{L} = \{\alpha_0, \dots, \alpha_n\}$ of $\mathbb{F}_{2^m}$ in random order. The public key is the standard form of a binary parity-check matrix for the public code denoted $\mathbf{H}_{\text{pub}} = (\mathbf{I}_{mt} \mid \mathbf{T})$, where $\mathbf{I}_{mt}$ is the identity matrix. Finding the polynomial $g(x)$ and the elements in $\mathcal{L}$ from the public parity-check matrix boils down to recovering the private key.

In the encapsulation step, a cipher-text is computed, which is a syndrome vector $\mathbf{H}_{\text{pub}}\mathbf{e}$, where $\mathbf{e}$ is a binary vector of Hamming weight t . The session key is the hash of the cipher-text together with the vector $\mathbf{e}$. The decapsulation step begins by computing the syndrome vector using the private key. More exactly, $\mathbf{s} = \mathbf{H}_{\text{priv}}\mathbf{v}^T$, where $\mathbf{v}$ is the cipher-text and $\mathbf{H}_{\text{priv}} = ((\alpha_i^j g^{-2}(\alpha_i))_{0 \leq j \leq 2t-1})_{0 \leq i \leq n-1}$ is the private parity-check matrix. Then, the Berlekamp-Massey (BM) algorithm [Ber15, Mas69] is used with input $\mathbf{s}$ to compute the error-locator polynomial $\sigma(x)$, which admits t roots satisfying $\sigma(\alpha_i) = 0 \leftrightarrow e_i = 1$ where α_i is the element of the Goppa support at index i . Finally, one determines the roots of $\sigma(x)$ and computes the initial vector $\mathbf{e}$.

Breaking the confidentiality of the scheme resumes to finding $\mathbf{e}$ from the cipher-text $\mathbf{v}$. This would enable the adversary to compute the session key. On the other side a key recovery attack would enable one to compute $g(x)$ and $\mathcal{L}$ and therefore, find all the session keys derived from $g(x), \mathcal{L}$.

1.3 Side-channel attacks against *Classic McEliece*

The growing interest for practical security aspects enabled various side-channel cryptanalysis. A summary of existing side-channel attacks against the session keys/private keys is illustrated in Table 1.

Although side-channel attacks against code-based public-key encryption schemes exist long time before the standardization process started (see [DRBL18] for a complete list of such attacks until 2016), we will focus our analysis only on those aiming the future

Table 1: Power-consumption side-channel attacks and Laser fault injections (grey cells) against *Classic McEliece*.

Step	Algorithm	Key recovery	Message recovery	Cipher-text assumptions
Key gen	Gauss. Elim.	[BCM ⁺ 25]		$\text{wt}(e) = 1$
Encaps	Matrix-vector mult.		[CCD ⁺ 21] [CDCG22] [GCCD23] [CGCD25] [BDK ⁺ 25]	
Decaps	load function FFT - root FFT - root root & check synd. comp.	[SCD ⁺ 23] [GJJ22a] [PGD ⁺ 22] [DCV ⁺ 24] [VCC ⁺ 25]	[LNPS20]	$\text{wt}(e) = 1$

standard. Let us briefly enumerate existing side-channel attacks going from key generation to encapsulation and finally decapsulation.

- In the **key generation step**, a private key recovery attack exploiting the Gaussian elimination is performed in [BCM⁺25]. Up to date, this is the only side-channel attack on the key generation step.
- The **encapsulation step** was a bit more scrutinized. The main operation in the encapsulation step is the matrix-vector multiplication. A laser fault injection attack was proposed in [CCD⁺21]. One year later a side-channel attack using a single observation of the power consumption during the matrix-vector multiplication was performed in [CDCG22]. The later was enhanced in [GCCD23]. In [CGCD25] a correlation attack was proposed, while in [BDK⁺25] an enhanced information set decoding with side-channel hints was performed, both retrieving the ephemeral key.
- Finally, the **decapsulation step** is the most studied step of this KEM from a side-channel perspective. In [SCD⁺23], the authors use a profiled attack against the load function. They managed to retrieve only partially the private key, since the information leaked when loading the coefficients of the secret Goppa polynomial was not sufficient for completely breaking the scheme. In [LNPS20] the authors showed that the session key can be recovered by side-channel attacks with multiple observations of the FFT. Exploiting the same algorithm was recently proposed for a complete private key recovery in [GJJ22a]. The attack fall under the chosen-cipher-text model where one exploit invalid cipher-texts during the decapsulation step. Using a similar setting a key-recovery fault injection attack was revealed in [PGD⁺22]. The fault injection is performed during the error locator polynomial computation, as well as the validity check. The syndrome computation was targeted in [DCV⁺24, VCC⁺25] where a profiled side-channel attack revealed the complete private key of the KEM.

1.4 Countermeasures

Since the decapsulation step represents one of the most common and practical attack target, we shall focus on this part of the *Classic McEliece* KEM. In the past, three vulnerable steps were identified (see Table 2 for state-of-the-art side-channel attacks together with

our attack). Let us briefly explain the possible countermeasures across existing attack steps.

Table 2: Key-recovery power-based side-channel attacks on the decapsulation algorithm of the *Classic McEliece* KEM with countermeasures and our attack

Ref	Step	Target	Attack scenario	Traces	Attack complexity	Countermeasure
[SCD ⁺ 23]	LOAD	$g(x)$	Template $\text{wt}(g_i)$	1	Exponential	SHUFFLE(LOAD)
[DCV ⁺ 24]	SYND	$g(x), \mathcal{L}$	Template $\text{wt}(\alpha_i^j g(\alpha_i)^{-2})$	1	Polynomial	SHUFFLE($\mathbf{H}_{\text{priv}} \mathbf{v}^T$)
This article	BM	$\mathcal{L}$	Template α_i	n	Polynomial	–
[GJJ22a]	ROOT	$\mathcal{L}$	Template α_i	n	Polynomial	Check $\sigma(x)$

1. Step LOAD: Load into memory the private polynomial $g(x)$ ([SCD⁺23]). Here the complexity of the attack is exponential (the authors recover only the Hamming weight of the coefficients of $g(x)$) and thus to recover the key remains unfeasible in practice. However, particular cases, known as weak keys were completely broken. A possible countermeasure is to shuffle the order in which coefficients are loaded. This would increase exponentially the number of possibilities for $g(x)$.
2. Step SYND: Compute the syndrome vector $\mathbf{H}_{\text{priv}} \mathbf{v}^T$. The attack in [DCV⁺24] exploits power leakage of the matrix-vector multiplication and manage to recover (using a template attack) the Hamming weight of the private matrix entries. The correct sequence of Hamming weights for each column in $\mathbf{H}_{\text{priv}}$ was a distinguisher for the private Goppa support and Goppa polynomial. The authors in [DCV⁺24] proposed as countermeasure to compute the multiplication using a classical shuffling technique. Also, an FFT implementation would prevent such attacks [CC21].
3. Step ROOT: Compute the roots of $\sigma(x)$. Guo et al. [GJJ22b] attack this step using plaintexts of Hamming weight 1 which map into $\sigma(x) = x - \alpha_i$, hence exploiting the leakage depending on a particular element α_i . To counter such attacks, a check instruction based on the results in [DCCR13] can be implemented. In [DCCR13] it was shown that the error-locator polynomials have few zero coefficients with high probability. Hence, to avoid such attacks one can impose $t - \delta$ non-zero coefficients, where δ is a small constant.

In Listing 1 we provide a possible solution for a secure implementation of the main steps in the decapsulation process based on existing works. For the SECURE-SHUFFLE one can use [VCMKS12], while the FFT solution is provided in [CC21]. All the steps that we intent to securely protect are those that were previously shown to be vulnerable to power-consumption side-channel attacks.

Listing 1: A secure decapsulation implementation of *Classic McEliece* KEM

```

1 int decrypt(unsigned char *e, const unsigned char *sk, const unsigned
  char *c)
2 {
3     ...
4     //load the private key
5     for (i = 0; i < SYS_T; i++) {
6         g[i] = secure_shuffle_load_gf(sk);
7         sk += 2;
8     }
9     g[ SYS_T ] = 1;
10    support_gen(L, sk);
11
12    //compute syndrome
13    //secure_shuffle_synd(s, g, L, r); or
14    fft_synd(s, g, L, r);
15
16    bm(locator, s);
17    //verify that locator is a not valid non-sparse polynomial
18    check_valid(locator, delta);
19
20    //the valid locator is evaluated
21    root(images, locator, L);
22    ...
23 }

```

Notice that two important steps have not been previously analyzed in the context of side-channel attacks: BM and SUPPORT_GEN. Hence there were no countermeasures proposed until now. We choose here to focus on BM since it is a generic decoding algorithm (can also be applied to Alternant codes, Generalized Reed-Solomon codes).

1.5 Contributions

Our article provides several contributions:

- We provide the first key-recovery side-channel attack on *Classic McEliece* KEM reference software implementation targeting the BM algorithm.
- We identify a side-channel leakage in the BM computation when small Hamming weight plaintexts are encrypted. In particular, we demonstrate and conduct a complete full key recovery attack on the MCELIECE348864. Experimentally, we validate the attack using the ChipWhisperer-Lite platform with the ARM Cortex-M4 microcontroller.
- We design an efficient template attack which recovers all secret values α , using a single attack trace per α .
- We show that possible countermeasures introduce significant and undesirable overhead. Furthermore, we show how to extend our attack to target Hamming weight 2, rendering such countermeasures impractical. Thus, our side-channel attack remains effective against current state-of-the-art countermeasures, requiring deeper investigation for side-channel resilient implementations.

2 Classic McEliece

2.1 Error correcting codes

Before we bring into discussion the required coding theory concepts let us enumerate the conventions and notations used through this paper.

We will denote integers using lowercase letters, while intervals of integers by $\llbracket a; b \rrbracket$. Sets will be denoted by italic upper cases. Matrices and vectors will be denoted using bold uppercase letters, and bold lowercase letters, respectively. The support of a vector will be denoted by $\text{sup}(\mathbf{v})$, and represents the set of positions where $v_i \neq 0$. A sub-matrix of a matrix $\mathbf{G}$ indexed by a set of rows $\mathcal{I}$ and a set of columns $\mathcal{J}$ is denoted by $\mathbf{G}[\mathcal{I}, \mathcal{J}]$. A row or a column of $\mathbf{G}$ is denoted by $\mathbf{G}[i :]$ and $\mathbf{H}[:, j]$, respectively.

Finite fields of order q will be denoted by $\mathbb{F}_q$, in particular $\mathbb{F}_{2^m}$ is the extension of $\mathbb{F}_2$ using an irreducible polynomial of degree m . Also, we will consider the mapping $\mathbb{F}_{2^m} \rightarrow \mathbb{F}_2^n$ defined by the irreducible polynomial $f(x)$ that generates the extension $\mathbb{F}_{2^m} \sim \mathbb{F}_2/f(x)$.

We can now move to coding theory requirements. $\mathcal{C}$ is a linear code of length n and dimension k if $\mathcal{C} \subseteq \mathbb{F}_q^n$ is a vector subspace of dimension k . The usual notation for it is $[n, k]_q$ -code. Any vector $\mathbf{c} = (c_0, \dots, c_{n-1}) \in \mathcal{C}$ is called a codeword.

Any code $\mathcal{C}$ can be defined using a basis, most commonly called *generator matrix* ($\mathbf{G}$) for $\mathcal{C}$, with $\text{Rank}(\mathbf{G}) = k$. We say that $\mathbf{G}$ is in standard form if $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{V})$.

Let $\mathbf{v}$ be a vector in $\mathbb{F}_q^n$. The Hamming weight of $\mathbf{v}$ is $\text{wt}(\mathbf{v}) = \#\{i \in \llbracket 0; n-1 \rrbracket \mid v_i \neq 0\}$, i.e., the cardinality of its support.

Let $\mathcal{C}$ be a $[n, k]_q$ linear code and $\mathbf{H}$ be an $(n - k) \times n$ matrix such that

$$\forall \mathbf{c} \in \mathcal{C} \Leftrightarrow \mathbf{H}\mathbf{c}^T = 0.$$

Then $\mathbf{H}$ is called a parity-check matrix of $\mathcal{C}$. Any $\mathbf{v} \in \mathbb{F}_q^n$ has a syndrome with respect to $\mathbf{H}$ which equals $\mathbf{s} = \mathbf{H}\mathbf{v}^T$.

The code family used in the *Classic McEliece* belongs to algebraic codes class and was invented by Valery Goppa [Gop70].

Definition 1 (Goppa Code). Let $\mathcal{L} = \{\alpha_0, \dots, \alpha_{n-1}\}$ with $\alpha_i \in \mathbb{F}_{2^m}$ and $\alpha_i \neq \alpha_j$ for $i \neq j$. Let $g(x) \in \mathbb{F}_q[x]$, $\deg(g) = t$ such that $\forall \alpha \in \mathcal{L}, g(\alpha) \neq 0$. Then the set

$$\mathcal{G}(g, \mathcal{L}) = \left\{ \mathbf{c} \in \mathbb{F}_2^n : \sum_{i=0}^{n-1} \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)} \right\}$$

is called a Goppa code with parameters $\mathcal{L}$ and $g(x)$.

Hence, a Goppa code is defined by two parameters, $g(x)$ – the Goppa polynomial, and $\mathcal{L}$ – Goppa support. A Goppa code is an $[n, k = n - mt]_2$ -code.

2.2 Decoding Goppa codes using Berlekamp-Massey

One can decode up to t errors in $\mathcal{G}(g, \mathcal{L})$ using the Patterson algorithm [Pat75]. However, since this strategy is difficult to implement in constant-time [Ber24] other decoders are preferred. For example, Berlekamp-Massey [Ber15, Mas69], or the interpolation proposed by D. J. Bernstein [Ber24] are using $\mathcal{G}(g^2, \mathcal{L})$ to decode t errors.

Here, we will focus on the BM algorithm 1. The decoding problem refers to recovering the vector $\mathbf{e} \in \mathbb{F}_2^n$ from $\mathbf{z} = \mathbf{c} + \mathbf{e}$ assuming $\text{wt}(\mathbf{e}) \leq t$ and $\mathbf{c} \in \mathcal{G}(g^2, \mathcal{L})$. We skip the details of why $\mathcal{G}(g^2, \mathcal{L}) = \mathcal{G}(g, \mathcal{L})$. First thing, we need to compute the syndrome vector/polynomial of $\mathbf{z}$ with respect to a parity-check matrix of $\mathcal{G}(g^2, \mathcal{L})$, say $\mathbf{H}_{g^2, \mathcal{L}}$, i.e., $\mathbf{s} = \mathbf{H}_{g^2, \mathcal{L}}\mathbf{z}$, $\mathbf{s} \in \mathbb{F}_2^{2t}$, where $\mathbf{H}_{g^2, \mathcal{L}} = \mathbf{H}_{\text{priv}}$. Since $\mathbf{H}_{g^2, \mathcal{L}}\mathbf{z} = \mathbf{H}_{g^2, \mathcal{L}}\mathbf{e}$ this implies that $s_j = \sum_{i \in \text{sup}(\mathbf{e})} g^{-2}(\alpha_i) \alpha_i^j$ for all $j \in \llbracket 0; 2t-1 \rrbracket$. The syndrome vector defines a syndrome polynomial

$$S(x) = \sum_{j=0}^{2t-1} s_j x^j = \sum_{j=0}^{2t-1} \left(\sum_{i \in \text{sup}(\mathbf{e})} g^{-2}(\alpha_i) \alpha_i^j \right) x^j. \quad (1)$$

Algorithm 1 Berlekamp-Massey Algorithm**Input:** A sequence $s_0, \dots, s_{2t-1}$ **Output:** The connection polynomial $C(x)$

```

1:  $L \leftarrow 0$  ▷ Degree of  $C(x)$ 
2:  $C(x) \leftarrow 1$ 
3:  $B(x) \leftarrow 1$ 
4:  $b \leftarrow 1$ 
5:  $l \leftarrow 1$ 
6: for  $N = 0$  to  $2t - 1$  do
7:    $d \leftarrow s_N + \sum_{j=1}^L C_j s_{N-j}$ 
8:   if  $d = 0$  then
9:      $l \leftarrow l + 1$ 
10:  else
11:     $T(x) \leftarrow C(x)$ 
12:     $C(x) \leftarrow C(x) - db^{-1}x^l B(x)$ 
13:    if  $2L \leq N$  then
14:       $L \leftarrow N + 1 - L$ 
15:       $B \leftarrow T$ 
16:       $b \leftarrow d$ 
17:       $l \leftarrow 1$ 
18:    else
19:       $l \leftarrow l + 1$ 
20: return  $C(x)$ 

```

The value of $S(x)$ has another well known form. Indeed, we have $(x - \alpha_i)^{-1} = \left(\frac{g^2(\alpha_i) - g^2(x)}{x - \alpha_i} \right) g(\alpha_i)^{-2}$ [MS77], which implies

$$S(x) = \sum_{i \in \text{sup}(\mathbf{e})} \frac{1}{x - \alpha_i} \pmod{g^2(x)} \quad (2)$$

where $S(x) = s_0 + s_1x + \dots + s_{2t-1}x^{2t-1}$. This implies, after some simplifications

$$S(x) = \frac{\sigma(x)'}{\sigma(x)} \pmod{g^2(x)} \quad (3)$$

where $\sigma = \prod_{i \in \text{sup}(\mathbf{e})} (x - \alpha_i)$ and $\gcd(\sigma, \sigma') = 1$. To find σ from $S(x)$ one needs to have $\deg(S) \geq 2t$ if the BM algorithm is applied.

Indeed, the BM algorithm computes the *connection polynomial* of an LFSR sequence. Suppose we are given a sequence of feedback coefficients $c_1, \dots, c_L$ and an initial state $s_0, \dots, s_L$. The LFSR sequence is the output of the linear recursion

$$s_j = - \sum_{i=1}^L c_i s_{j-i} \quad \text{for } j = L, L+1, \dots \quad (4)$$

The feedback coefficients define the connection polynomial, $C(x) = 1 + c_1x + \dots + c_Lx^L$, with $C(x) = 1$ for $L = 0$. The polynomial C is sequentially constructed using the values s_i as illustrated in 1.

In [Mas69], equation 17 and later in Theorem 4, it is demonstrated that any LFSR sequence with connection polynomial $C(x)$ satisfies an algebraic equation $S(x) = \frac{D(x)}{C(x)}$ where $D(X)$ can be calculated using linear algebra. In the case of binary Goppa codes by (3) σ is the connection polynomial computed by BM with input $S(x)$.

2.3 The *Classic McEliece* KEM

The parameters proposed by the authors of the *Classic McEliece* for a security level of at least 128-bits are as follows. In the NIST submission a first set of parameters considers $m = 12, n = 3488, t = 64$ (MCEliece348864). Other parameter sets are used both in the NIST and ISO submission and are defined as follows. The extension $\mathbb{F}_{2^m}$ is $\mathbb{F}_2/f(x)$ where $f(x) = x^{13} + x^4 + x^3 + x + 1$, hence $m = 13$. There are 3 distinct code lengths considered, each with a corresponding decoding parameter: $n = 6688, t = 128$; $n = 6960, t = 119$; $n = 8192, t = 128$.

The KEM is composed of three algorithms, Key Generation 2, Encapsulation 3, and Decapsulation 4.

Algorithm 2 Key generation – *Classic McEliece*

Input: (m, n, t)

Output: The private key $\text{sk} = (g, \mathcal{L})$ and the public key $\text{pk} = \mathbf{T}$

- 1: Generate a random subest $\mathcal{L} = \{\alpha_0, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_{2^m}$
 - 2: Generate a random irreducible monic polynomial $g \in \mathbb{F}_{2^m}[x]$ with $\deg(g) = t$
 - 3: Compute $\mathbf{H} = \left((\alpha_i^j g^{-1}(\alpha_i))_{j \in [0; t-1]} \right)_{i \in [0; n-1]}$
 - 4: Transform $\mathbf{H}$ a binary matrix $\mathbf{H}'$
 - 5: Compute the standard-form of $\mathbf{H}'$: $\mathbf{H}_{\text{pub}} = (\mathbf{I}_{mt} \mid \mathbf{T})$
 - 6: **return** $\text{sk} = (g, \mathcal{L})$ and $\text{pk} = \mathbf{T}$
-

Breaking the *Classic McEliece* key resumes to solving the Goppa code equivalence problem. Indeed, given $\mathbf{H}_{\text{pub}}$ it is considered difficult to compute $g(x), \mathcal{L}$. First of all, we do not know which elements from $\mathbb{F}_{2^m}$ are in $\mathcal{L}$ and even in the case $\mathcal{L} = \mathbb{F}_{2^m}$ we do not know the order of the elements of $\mathbb{F}_{2^m}$ in $\mathcal{L}$. Secondly, there are roughly as many Goppa codes as possible polynomials g , number that scales asymptotically as $O(2^{mt}/mt)$ [LS01]. Hence, a possible solution would be to 1) enumerate all possible g and 2) find $\mathcal{L}$ with the knowledge of g using the Support Splitting Algorithm [Sen00]. Having partial information regarding the private key substantially decreases the complexity of the key recovery procedures, fact that will be detailed in a next section.

3 Attack methodology

3.1 Chosen plaintext model

We will use chosen plaintexts such that $\text{wt}(\mathbf{e}) = 1$, thus isolating each unknown element $\alpha_i \in \mathcal{L}$. For such plaintexts the syndrome value will be equal to $\mathbf{s} = (g^{-2}(\alpha_i), \dots, g^{-2}(\alpha_i)\alpha_i^{2t-1})$. The reason for doing so is that computations in the Berlekamp-Massey algorithm will mainly depend on the isolated element α_i . Indeed, recall that the discrepancy is computed at each step using the formula

$$d = s_N + \sum_{j=1}^L C_j s_{N-j}. \quad (5)$$

Hence, by computing the value d one verifies whether the connection polynomial $C(x)$ generates the sequence $s_0, s_1, \dots, s_N$. We have detailed the computations in the Berlekamp-Massey decoder in Table 3. Notice that except for the first two iteration steps, the discrepancy equals zero.

The interesting part for our side-channel attack is not so much that the value equals zero, but the fact that when it is calculated each time one multiplies α_i with a syndrome value

Algorithm 3 Encapsulation – *Classic McEliece***Input:** $\text{pk} = T$ **Output:** A ciphertext z and a session key K

- 1: Generate a random vector $e \in \mathbb{F}_2^n$ with $\text{wt}(e) = t$
- 2: Compute $z = (I_{mt} \mid T) e^T$
- 3: Compute $K = \text{hash}(1|e|z)$
- 4: **return** z and K

Algorithm 4 Decapsulation – *Classic McEliece***Input:** z and $\text{sk} = (g, \mathcal{L})$ **Output:** The session key K

- 1: Pad z with $n - mt$ zeros to obtain: $v = (z, 0, \dots, 0)$
- 2: Compute $H_{\text{priv}} = \left((\alpha_i^j g^{-2}(\alpha_i))_{j \in \llbracket 0; 2t-1 \rrbracket} \right)_{i \in \llbracket 0; n-1 \rrbracket}$
- 3: Compute $s = H_{\text{priv}} v^T$
- 4: Compute $\sigma(x)$ using the BM algorithm
- 5: Recover e from σ
- 6: Compute $K = \text{hash}(1|e|z)$
- 7: **return** K

Table 3: Computations in the Berlekamp-Massey algorithm for a plaintext with $\text{wt}(e) = 1$ for the *Classic McEliece* KEM

Iter. (N)	syndrome components	Discrepancy (d)	C	b
Init			$[1]$	1
$N = 0$	$[g(\alpha_i)^{-2}]$	$d = s_0 = g(\alpha_i)^{-2}$	$[1, g(\alpha_i)^{-2}]$	$g(\alpha_i)^{-2}$
$N = 1$	$[g(\alpha_i)^{-2}, g(\alpha_i)^{-2}\alpha_i]$	$s_1 + s_0 C_1 = g(\alpha_i)^{-2}\alpha_i + g(\alpha_i)^{-2}g(\alpha_i)^{-2}$	$[1, g(\alpha_i)^{-2} + d/b] = [1, \alpha_i]$	$g(\alpha_i)^{-2}$
$N = 2$	$[g(\alpha_i)^{-2}\alpha, g(\alpha_i)^{-2}\alpha_i^2]$	$s_2 + s_1 C_1 = g(\alpha_i)^{-2}\alpha_i^2 + g(\alpha_i)^{-2}\alpha_i \alpha_i = 0$	$[1, \alpha_i]$	$g(\alpha_i)^{-2}$
$N = 3$	$[g(\alpha_i)^{-2}\alpha^2, g(\alpha_i)^{-2}\alpha_i^3]$	$s_3 + s_2 C_1 = g(\alpha_i)^{-2}\alpha_i^3 + g(\alpha_i)^{-2}\alpha_i^2 \alpha_i = 0$	$[1, \alpha_i]$	$g(\alpha_i)^{-2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$N = i$	$[g(\alpha_i)^{-2}\alpha^{i-1}, g(\alpha_i)^{-2}\alpha_i^i]$	$s_i + s_{i-1} C_1 = g(\alpha_i)^{-2}\alpha_i^i + g(\alpha_i)^{-2}\alpha_i^{i-1} \alpha_i = 0$	$[1, \alpha_i]$	$g(\alpha_i)^{-2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

$s_{i-1} = g(\alpha_i)^{-2}\alpha^{i-1}$ and then it adds the result to another syndrome value $s_i = g(\alpha_i)^{-2}\alpha_i^i$. Hence, in each one of the $2t - 2$ iteration steps the value α_i is multiplies with distinct values from $\mathbb{F}_{2^m}$ (the syndrome components). It is thus the only apparent invariant in the computations.

Typically in this model, one expects to determine the value α_i which corresponds to the position i based on the power consumption of the Berlekamp-Massey algorithm when evaluated on the syndrome corresponding to the chosen message e with $e_i = 1$ and $e_j = 0$ for all $j \neq i$. Repeating this procedure for each position $i \in \llbracket 0; n-1 \rrbracket$ recovers the entire support $\mathcal{L}$.

3.2 Breaking Goppa codes with hints

Suppose now that we are able to find the Goppa support $\mathcal{L}$. The next step would be to compute the remaining part of the private key, namely the Goppa polynomial. For such a task, a well-known technique exists [KM23]:

1. Compute the generator matrix G_{pub} from H_{pub}
2. Generate a non-zero codeword c using G_{pub}

3. Compute a possible multiple of g using

$$f(x) = \sum_{i \in \text{sup}(\mathbf{c})} \prod_{j \in \text{sup}(\mathbf{c}), j \neq i} (x - \alpha_j)$$

4. Factor $f(x)$ into irreducible polynomials and select those of degree t .
5. Test which of the polynomials from the previous step are valid.

The last step can be done in several ways, the authors in [KM23] proposed to compute all syndromes corresponding to the row vectors of $\mathbf{G}_{\text{pub}}$ and check whether they are equal to 0 mod $f_i(x)$ (where f_i is a factor of f of degree t). Another possible way is to simply reconstruct the parity-check matrix with $\mathcal{L}$ and f_i and verify whether it is equal to $\mathbf{H}_{\text{pub}}$.

Although here we obtain information about the Goppa points, in the literature other side-channel information leading to reconstruction of the Goppa polynomial exist [DCV⁺24]. In Table 4 we illustrate state-of-the-art solutions for computing the complete private key of the *Classic McEliece* KEM with extra knowledge.

Table 4: Solutions for computing the private key of the *Classic McEliece* KEM using partial knowledge of the secret ($g(x)$ or $\mathcal{S} \subseteq \mathcal{L}$).

Knowledge	Complexity	Reference
$g(x)$	$\mathcal{O}(n^3 + 2^h n^2 \log n)$	[Sen00]
$\#\mathcal{S} = mt + 1$	$\mathcal{O}(n^5)$	[KM23, Thm. 4]
$\mathcal{S} = \mathcal{L}$	$\mathcal{O}(n^3)$	[KM23, Thm. 1]
$\#\mathcal{S} = tm + 1$ and $g(x)$	$\mathcal{O}(n^4)$	[KM23, Thm. 3]
$\#\mathcal{S} = tm + \delta$ and $\{g(\alpha_i) \mid \alpha_i \in \mathcal{S}\}$	$\mathcal{O}(n^3)$	[DCV ⁺ 24]

Hence, one can either retrieve all Goppa points α_i and apply $\mathcal{O}(n^3)$ or retrieve $mt + 1$ and compute $g(x)$ and remaining points in $\mathcal{O}(n^5)$.

3.3 Leakage Model

Our targets are the secret coefficients α_i of the private Goppa set $\mathcal{L} = \{\alpha_0, \dots, \alpha_{n-1}\}$ with $\alpha_i \in \mathbb{F}_{2^m}$. As explained in Section 3.1, we consider only vectors $\mathbf{e}$ such that $\text{wt}(\mathbf{e}) = 1$. Hence, during the computation of the Berlekamp-Massey algorithm, there is only one secret coefficient α_i that is being processed and each such coefficient has m secret bits (with $m = 12$ or $m = 13$, depending on the implementation).

In power-based [KJJ99] or EM-based [QS01, GMO01] side-channel attacks, a typical assumption is that a given instruction that operates on some operand a leaks its Hamming Weight $\text{wt}(a)$ [BCO04]. Furthermore, it is also a common observation that besides the leakage on the operand, the leakage data is influenced by noise (due to other operations in the circuit, environmental noise, etc.), which can usually be modeled using a Gaussian distribution $\mathcal{N}(0, \sigma)$. Generalizing the operand a to some function f of the value a , we can write that the leakage of some particular instruction processing operand a leaks as:

$$l(a) = \text{wt}(f(a)) + \mathcal{N}(0, \sigma)$$

We note that although the Hamming Weight leakage model is commonly used in side-channel attack evaluations, there have been some works demonstrating that we may obtain all the bits of the target operand from single instructions, when we have enough profiling data and access to several executions of the target instruction [CK14, CK17]. Therefore, we could generalize the leakage of a given instruction as:

$$l(a) = lm(f(a)) + \mathcal{N}(0, \sigma),$$

where lm represents some arbitrary leakage model for $f(a)$.

We make the additional remark that many side-channel attack evaluations consider a chosen-plaintext attack scenario, in which the target function depends on two values: a fixed but secret value (e.g. a key word) and a variable but known value (e.g. a plaintext word). However, in our context, we are faced with only a fixed secret value (the α_i secret coefficients) that is processed during the Berlekamp-Massey algorithm. Therefore, the leakage function in our context can generally be modeled as

$$l(\alpha_i) = lm(f(\alpha_i)) + \mathcal{N}(0, \sigma).$$

The following important observation is that the Berlekamp-Massey algorithm requires many instructions (see Table 3) that operate on the same secret coefficient α_i . Considering the above observations and the fact that during a side-channel attack we may obtain a variable number of leakage samples for each instruction, we can state that during the execution of the Berlekamp-Massey algorithm, we obtain leakage traces $\mathbf{x}_i \in \mathbb{R}^{n_s}$, $\mathbf{x}_i = \{\mathbf{x}_{i1}, \mathbf{x}_{i2}, \dots, \mathbf{x}_{in_s}\}$, where each leakage sample $\mathbf{x}_{ij}$ in the trace can be modeled as¹:

$$\mathbf{x}_{ij} = lm(f_j(\alpha_i)) + \mathcal{N}(0, \sigma_j).$$

3.4 TVLA

To evaluate whether the Berlekamp-Massey algorithm leaks information about the secret support values α_i , we performed a fixed-versus-random Test Vector Leakage Assessment (TVLA) [GGJR⁺11] over the Berlekamp-Massey execution.

This statistical test aims to determine whether the power consumption during the computation of a fixed error-locator polynomial $\sigma_f(x)$ by Berlekamp-Massey exhibits a distinguishable pattern compared to that of a randomly generated polynomial $\sigma_r(x)$, where $\alpha_f \neq \alpha_r$. The power traces recorded during the computation of $\sigma_f(x)$ are referred to as *fixed traces*, while those corresponding to $\sigma_r(x)$ are denoted as *random traces*. These will be grouped into the *fixed set*, $\mathcal{T}_f$, and the *random set*, $\mathcal{T}_r$, respectively. A Welch's t-test is applied point-wise across the two sets to test the null hypothesis that both sets are drawn from the same distribution (i.e., no leakage is detected):

$$t_{obs} = \frac{\hat{\mu}_f - \hat{\mu}_r}{\sqrt{\frac{s_f^2}{n_f} + \frac{s_r^2}{n_r}}},$$

where:

- $\hat{\mu}_f, \hat{\mu}_r$: mean values at a given sample point computed across all traces from $\mathcal{T}_f, \mathcal{T}_r$
- s_f, s_r : standard deviations at a given sample point computed across all traces from $\mathcal{T}_f, \mathcal{T}_r$
- $n_f = |\mathcal{T}_f|$ and $n_r = |\mathcal{T}_r|$

A threshold of $|t_{obs}| \geq 4.5$ is typically used, corresponding to a confidence level of 99.998%. If this threshold is exceeded at any sample point, the implementation is said to exhibit leakage at that point.

¹For generality, we assume that for each sample $\mathbf{x}_{ij}$ the corresponding leakage function f_j can be different, e.g. due to manipulation of different bits, using different hardware components, etc.

3.5 Classification using Template Attacks

Recall that our context is that of a chosen plaintext/ciphertext attack, in which we additionally allow the attacker to use side-channel traces to determine the secret coefficients α_i of the private Goppa set $\mathcal{L} = \{\alpha_0, \dots, \alpha_{n-1}\}$ with $\alpha_i \in \mathbb{F}_{2^m}$.

In this scenario, considering that the side-channel leakage only depends on the secret coefficients α_i processed during the Berlekamp-Massey algorithm, the most suitable side-channel attack is based on splitting the attack into two parts: a profiling step and an attack (or classification) step. This assumes that an adversary is allowed to profile the side-channel traces using a similar device as the one used during the attack step. Such assumption is widely accepted, given that most of the hardware devices on which such encryption algorithms are expected to run (e.g. IoT devices, automotive controllers, smartphones) are readily available. Furthermore, for security critical devices that may be more difficult to obtain on the market, it is still required to perform a security evaluation such as Common Criteria, considering the strongest possible attacks, and one such powerful attack is precisely this one in which we assume the adversary to possess a copy of the target device, for the profiling step.

Therefore, considering the setup in which an attacker or evaluator will use a device for the profiling step and another device for the attack/classification step, we are left with the choice of the method to perform the overall profiling and classification steps. In the context of side-channel attacks there are two main approaches: multivariate statistics (aka Template Attacks [CRR02, CK17]) and Machine Learning [HGDM⁺11, HGG20, PPM⁺23]. The former is known for being very efficient in contexts of linear dependencies and relatively fixed execution environments, while the latter is better suited for non-linear dependencies as well as contexts with jitter, desynchronisation or other possible countermeasures. In our context, given the well-known setup that we assume, as we want to explore a worst-case scenario, we use Template Attacks, described next.

3.5.1 Template Attacks

For each $\alpha \in \mathbb{F}_{2^m}$, we assume that we have obtained n_p leakage traces $\mathbf{x}_i^\alpha$, with $i = 1, \dots, n_p$. Then, we can compute the template parameters $\bar{\mathbf{x}}_\alpha \in \mathbb{R}^{n_s}$ and $\mathbf{S}_\alpha \in \mathbb{R}^{n_s \times n_s}$ for each α :

$$\bar{\mathbf{x}}_\alpha = \frac{1}{n_p} \sum_{i=1}^{n_p} \mathbf{x}_i^\alpha, \quad (6a)$$

$$\mathbf{S}_\alpha = \frac{1}{n_p - 1} \sum_{i=1}^{n_p} (\mathbf{x}_i^\alpha - \bar{\mathbf{x}}_\alpha)(\mathbf{x}_i^\alpha - \bar{\mathbf{x}}_\alpha)', \quad (6b)$$

where the sample mean $\bar{\mathbf{x}}_\alpha$ and the sample covariance matrix $\mathbf{S}_\alpha$ estimate the true mean $E(\mathbf{x}_\alpha)$ and true covariance $\text{Cov}(\mathbf{x}_\alpha)$ of the random vector $\mathbf{x}^\alpha$.

The sample mean $\bar{\mathbf{x}}_\alpha$ and sample covariance $\mathbf{S}_\alpha$ are *sufficient statistics*: they completely define the underlying distribution [JW⁺02, Chapter 4]. Then, the probability density function (pdf) of a leakage vector $\mathbf{x}$, given the parameters $\bar{\mathbf{x}}_\alpha$ and $\mathbf{S}_\alpha$ for each α , is

$$f(\mathbf{x} | \alpha) = \frac{1}{\sqrt{(2\pi)^{n_s} |\mathbf{S}_\alpha|}} \cdot e^{-\frac{1}{2}(\mathbf{x} - \bar{\mathbf{x}}_\alpha)' \mathbf{S}_\alpha^{-1} (\mathbf{x} - \bar{\mathbf{x}}_\alpha)}. \quad (7)$$

In the *attack* phase, we try to infer the secret value $\alpha^* \in \mathbb{F}_{2^m}$ processed by the attacked device. We obtain n_a leakage vectors $\mathbf{x}_i \in \mathbb{R}^{n_s}$ from the attacked device, using the same recording technique and compression method as in the profiling phase, resulting in the leakage matrix $\mathbf{X}_{\alpha^*} \in \mathbb{R}^{n_a \times n_s}$. Then, for each $\alpha \in \mathbb{F}_{2^m}$, we compute a *discriminant score* $D(\alpha | \mathbf{X}_{\alpha^*})$. Finally, we try all $\alpha \in \mathbb{F}_{2^m}$ on the attacked device, in order of decreasing score (optimized brute-force search, e.g. for a password or cryptographic key), until we find the correct α^* .

Using the available n_a leakage traces, we can compute the following log discriminant, derived from the joint log-likelihood [CK17]:

$$D_{\log}(\alpha \mid \mathbf{X}_{\alpha^*}) = -\frac{n_a}{2} \log |\mathbf{S}_{\alpha}| - \frac{1}{2} \sum_{i=1}^{n_a} (\mathbf{x}_i - \bar{\mathbf{x}}_{\alpha})' \mathbf{S}_{\alpha}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}}_{\alpha}). \quad (8)$$

Furthermore, a very useful observation is that generally the noise in the traces is independent of the target value α . In this case, we can combine the information from all the covariance matrices into a single *pooled* covariance:

$$\mathbf{S}_{\text{pooled}} = \frac{1}{|\mathbb{F}_{2^m}|(n_p - 1)} \sum_{\alpha \in \mathbb{F}_{2^m}} \sum_{i=1}^{n_p} (\mathbf{x}_i^{\alpha} - \bar{\mathbf{x}}_{\alpha})(\mathbf{x}_i^{\alpha} - \bar{\mathbf{x}}_{\alpha})', \quad (9)$$

leading to the very efficient *linear discriminant* [CK17]:

$$D_{\text{linear}}(\alpha \mid \mathbf{X}_{\alpha^*}) = \bar{\mathbf{x}}_{\alpha}' \mathbf{S}_{\text{pooled}}^{-1} \left(\sum_{i=1}^{n_a} \mathbf{x}_i \right) - \frac{n_a}{2} \bar{\mathbf{x}}_{\alpha}' \mathbf{S}_{\text{pooled}}^{-1} \bar{\mathbf{x}}_{\alpha}. \quad (10)$$

3.5.2 Compression methods (PCA and LDA)

The use of Template Attacks as presented above works well only for relatively short traces (e.g. for $n_s \leq 5000$), as the covariance matrices $\mathbf{S}_{\alpha}$ and even $\mathbf{S}_{\text{pooled}}$, as well as their inverses (required in the computation of the discriminants) require a very large storage capacity ($\mathcal{O}(n_s^2)$). Furthermore, the computation time is also heavily impacted by the number of leakage samples n_s in each trace.

To solve these issues, it is typical to use some compression method in order to reduce the size of the traces, while keeping most of the information. Two very useful compression methods in the context of Template Attacks are Principal Component Analysis (PCA) [APSQ06] and Linear Discriminant Analysis (LDA) [SA08]. Using these methods we can reduce a raw leakage trace of 100.000 samples to a leakage vector of only 10 samples, while keeping a very high attack efficiency, as we shall show in the evaluation section. For more details of using PCA and LDA with Template Attacks please refer to the detailed paper of Choudary and Kuhn [CK17].

4 Experimental platform

4.1 Hardware Setup

For the implementation of the attack, we selected the ChipWhisperer-Lite development board from NewAE Technology as the target platform, featuring a STM32F303RCT6 32-bit ARM Cortex-M4 microcontroller which serves as a representative example of real-world embedded systems commonly used for cryptographic operations. The firmware was compiled using the `arm-none-eabi-gcc` toolchain (version 6.3.1) with optimization level `-O3`, chosen to emulate practical deployment conditions. Additionally, the default configuration provided by the ChipWhisperer open-source API was modified to set the Cortex-M4's clock frequency to 45 MHz.

We performed the power trace acquisition using a PicoScope 6404D digital oscilloscope equipped with differential active probes for noise-resilient measurements. We used a sampling frequency of 1.25 GHz with onboard averaging downsampling, thereby improving

the signal-to-noise ratio while preserving relevant power leakage features. The ChipWhisperer's GPIO pins were configured to deliver trigger signals to the PicoScope for precise synchronization, while its dedicated measurement lines were used to capture the target device's power consumption.

4.2 Experimental Leakage Assessment

Algorithm 5 The TVLA trace acquisition algorithm

Input: A value $\alpha_f \in \mathbb{F}_{2^m}$ and a set of keys $\mathcal{K}$ satisfying (11)

Output: The fixed set $\mathcal{T}_f$ and random set $\mathcal{T}_r$ for TVLA

```

1: for  $1 \leq n_{iter} \leq 2n_t$  do
2:   if  $n_{iter} = 0 \pmod{2}$  then
3:      $ind \leftarrow f$   $\triangleright K = K_f \Rightarrow \mathcal{L}_K(i) = \alpha_f$ 
4:   else
5:      $ind \leftarrow r$   $\triangleright K = K_r \Rightarrow \mathcal{L}_K(i) \neq \alpha_f$ 
6:   Select  $K_{ind} \in \mathcal{K}$ 
7:   Compute  $\mathbf{e} \in \mathbb{F}_2^n$  with  $\text{wt}(\mathbf{e}) = 1$ ,  $e_i = 1$ 
8:   Compute  $\mathbf{z} = \mathbf{H}_{\text{pub}} \mathbf{e}^T$ 
9:   Send  $\mathbf{z}$  to the target device and capture the power trace  $t$  during BM
10:   $\mathcal{T}_{ind} \leftarrow \mathcal{T}_{ind} \cup \{t\}$ 

```

To begin, we selected a fixed value $\alpha_f \in \mathbb{F}_{2^m}$ and defined the desired number of traces for both the fixed and random sets, such that $|\mathcal{T}_f| = |\mathcal{T}_r| = n_t = 900$. We then generated a set of key pairs K , where $|K| \geq n_t$, and each key pair satisfies the condition that α_f is included in every secret support:

$$K = \{(\text{pk}_i, \text{sk}_i) \mid \alpha_f \in \mathcal{L}_i, \mathcal{L}_i \in \text{sk}_i\} \quad (11)$$

Once α_f has been selected and the key set K has been constructed, we proceed to generate the fixed and random trace sets by applying Algorithm 5. Note that at line 3 and 5, the plaintext e_i is chosen so that $\mathcal{L}(i) = \alpha_f$ for the fixed set (denoted here K_f), and $\mathcal{L}(i) \neq \alpha_f$ for the random set (denoted here K_r).

For the trace acquisition, the oscilloscope was configured with the sampling settings detailed in Section 4.3, i.e. 45 kHz, focusing exclusively on the execution window of the Berlekamp–Massey algorithm. After constructing the fixed and random trace sets, we performed a point-wise TVLA across the two sets. The results are presented in Figure 1. As illustrated in the figure, the resulting TVLA plot reveals statistically significant leakage, clearly exceeding the standard detection threshold of 4.5. This observation supports the hypothesis that the Berlekamp–Massey algorithm leaks information about the secret support values under our chosen-plaintext attack scenario.

4.3 Template Construction

Considering the TVLA results from Section 4.2, we focus our security evaluation on the execution of the Berlekamp–Massey algorithm, using the template attack described in Section 3.5.

During the profiling phase, 200 power traces were acquired for each possible secret value $\alpha \in \mathbb{F}_{2^m}$, resulting in a total of 819,200 traces. The position of α within the support vector $\mathcal{L}$ was varied for every power trace capture by using different keys. These keys were selected from a set $K = \{(\text{pk}, \text{sk})\}$ such that:

$$|\{(\text{pk}_i, \text{sk}_i) \mid \alpha_j \in \mathcal{L}_i, (\text{pk}_i, \text{sk}_i) \in K\}| \geq 200, \forall \alpha_j \in \mathbb{F}_{2^m}.$$

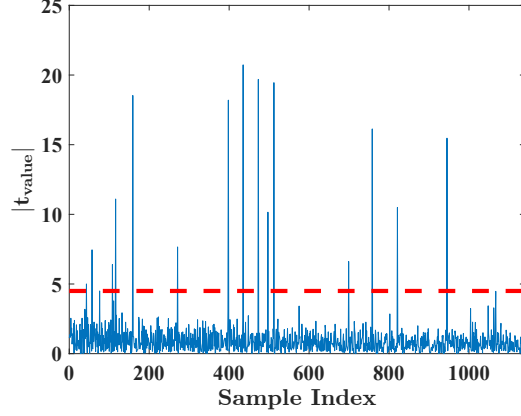


Figure 1: Point-wise TVLA over Berlekamp-Massey, sampled at 45 kHz.

Each trace exclusively captures the execution window of the Berlekamp–Massey algorithm within the decapsulation procedure and initially contained 114.000 samples. These were recorded by averaging every 280 original samples, corresponding to an effective sampling rate of 4.5 MHz. To further reduce data size and processing overhead, the traces were subsequently decimated in software by a factor of 100, yielding a final sampling rate of 45 kHz.

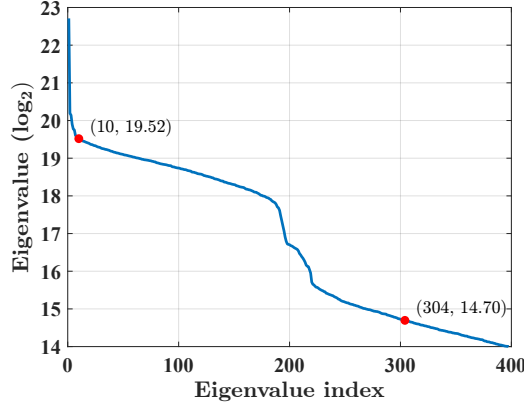


Figure 2: Reducing the dimension with LDA.

To reduce the dimensionality of the power traces while preserving the relevant discriminative features, we applied Linear Discriminant Analysis (LDA). To select the dimensionality for the reduced feature space, we analyzed the eigenvalue spectrum computed during LDA which reflects the discriminative power of each projection direction. As shown in Figure 2, the discriminative power of the components, indicated by their associated eigenvalues, decays sharply after the first few directions. Based on this plot, we visually selected a cutoff point by retaining the first $m_1 = 10$ components. Additionally, we applied the Cumulative Percentage of Total Variation (CPTV) criterion [Jol02]. This approach selects the smallest r such that the cumulative sum of the first r eigenvalues sorted in descending order accounts for at least a fixed fraction f of the total sum:

$$\frac{\sum_{1 \leq i \leq r} \lambda_i}{\sum_{1 \leq i \leq m} \lambda_i} \geq f$$

Using CPTV, we preserved the first $m_2 = 304$ components that account for 95% of the

eigenvalues' sum. Consequently, we evaluate the performance of the attack under both dimensionality settings: $m_1 = 10$ and $m_2 = 304$.

The collected traces were subsequently divided into two disjoint subsets: a training set used for the profiling phase and an attack set reserved for evaluation. To assess the impact of the profiling set size on attack performance, we conducted the experiments using training sets of 50, 100, and 150 traces. For each profiling set size, a range of attack configurations was explored by varying the LDA compression dimension and by using multiple numbers of attack traces n_{attack} for each secret value $\alpha \in \mathbb{F}_{2^m}$. Nonetheless, due to the significant gap between the two selected LDA dimensions ($m_1 = 10$ and $m_2 = 304$), the number of attack traces used with the higher dimension was intentionally kept smaller to reduce computational overhead. More precisely, when the LDA dimension is equal to 10, the number of considered attack traces is taken from $\{1, 2, 3, 4, 5, 10, 20, 30, 40, 50\}$, while for a dimension of 304 the set is $\{1, 2, 3, 4, 5\}$.

5 Attack implementation and evaluation

To evaluate the success of our attacks, we used the *empirical guessing entropy* [RPC22] (GE), i.e. the average *key rank* [SMY09] ($\overline{GE}$). The key rank is defined as the position of the correct key in the list of possible keys, sorted using the information from the Template Attacks; or alternatively, the key rank is the number of candidate keys that are more likely (according to the Template Attack) than the correct key. The lower the key rank, the better the attack performs.

To obtain the average key rank (empirical guessing entropy), we performed the attack 10 times for each secret value $\alpha \in \mathbb{F}_{2^m}$, using a different subset of attack traces in each repetition and covering all configurations outlined in Section 4.3. Let $GE(\alpha)$ be this empirical guessing entropy for each secret value α . We further averaged this empirical guessing entropy over all possible values of the secret value α , obtaining $\overline{GE}$.

We consider a perfect key recovery when we obtain an average guessing entropy $\overline{GE}$ such that $\overline{GE} \leq 0.004$. For simplicity, such values are approximated by 0.

5.1 Guessing entropy results

The success of our attack mainly depends on the number of profiling traces used to build the template, the dimension of the LDA as well as number of samples.

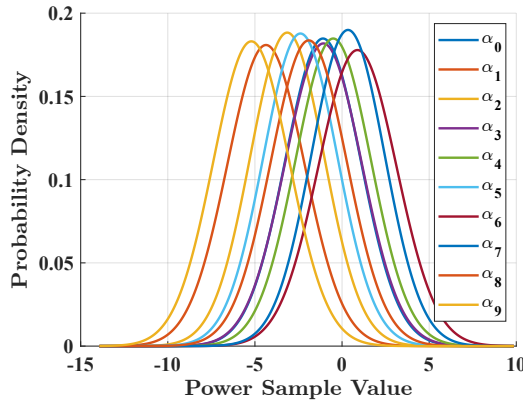
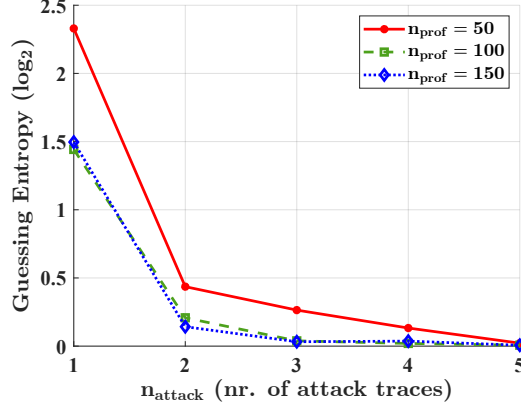


Figure 3: Distributions for the first 10 α_i values, using 1140-sample traces and LDA dimension 304.

When using the higher LDA dimension of 304 in conjunction with the traces of 1140

Figure 4: $\log_2(\overline{GE})$ for LDA dimension 10.

samples, our template attack consistently achieves perfect key recovery (i.e. $\overline{GE} = 0$) across all tested configurations, regardless of the number of profiling or attack traces. In other words, we are able to recover the entire secret Goppa support with a single trace per α . These observations are further confirmed by the good separation between the LDA leakage distributions of various α s, as illustrated in Figure 3.

When the LDA dimensionality is reduced to 10, the effectiveness of the attack becomes closely tied to the number of profiling and attack traces used, as summarized in the first three rows of Table 5 and also shown in Figure 4.

Table 5: Average guessing entropy in function of n_{attack} for using 50,100, and 150 profiling traces and two distinct sample rates per trace, for an LDA dimension equal to 10.

Samples	Profiling traces	n_{attack}									
		1	2	3	4	5	10	20	30	40	50
1140	50	2.330	0.436	0.264	0.133	0.022	0	0	0	0	0
	100	1.443	0.208	0.039	0.020	0.007	0	0	0	0	0
	150	1.497	0.142	0.032	0.037	0.007	0	0	0	0	0
114	50	0.282	0.157	0.115	0.095	0.081	0.055	0.047	0.042	0.039	0.038
	100	0.287	0.147	0.107	0.084	0.081	0.053	0.044	0.038	0.038	0.035
	150	0.275	0.148	0.110	0.088	0.076	0.055	0.043	0.040	0.039	0.038

As anticipated, increasing the number of attack traces consistently improves the attack performance. For each profiling configuration, the average guessing entropy $\overline{GE}$ shows a monotonically decreasing trend as more attack traces are used. Notably, perfect key recovery is achieved with as few as 10 attack traces in all high-sample (i.e. 1140 samples) scenarios. This demonstrates the efficiency of the template attack under sufficient attack data, even at low LDA dimensionality.

Additionally, for a fixed number of attack traces, increasing the profiling traces significantly enhances attack performance. A larger profiling set improves the statistical quality of the templates, resulting in lower $\overline{GE}$ values. This behavior reflects the expected inverse proportionality between the profiling trace count and the resulting guessing entropy.

Despite initially exceeding 1—and even surpassing 2 when only 50 profiling traces are used—the guessing entropy declines sharply as the number of attack traces increases. The reduction follows a steep slope that becomes even more pronounced as the number of profiling traces increases. This steeper decline indicates that the additional profiling data helps the model better discriminate between classes, allowing fewer attack traces to achieve low guessing entropy.

5.2 Reducing the number of samples

Given that an average guessing entropy of zero was achieved using traces with 1140 samples, the same attack methodology was applied to a second dataset containing traces with only 114 samples each, corresponding to a reduced sampling rate of 4.5 kHz. In this experiment, only an LDA dimension of 10 was considered. The results for this configuration are presented in the bottom half of Table 5.

Interestingly, the shorter traces exhibit surprisingly strong performance in low-data attack scenarios. When using a very small number of attack traces (e.g., $n_{\text{attack}} \leq 5$), the templates built from the 114-sample traces outperform those from the 1140-sample traces. In particular, when using just one attack trace, the guessing entropy is significantly lower with shorter traces, reaching values close to zero, even when profiling traces are limited to 50. In fact, under this constraint (i.e., using only 50 profiling traces), reduced-sample templates maintain a performance edge for $n_{\text{attack}} \leq 4$.

However, this advantage quickly diminishes as the number of profiling traces increases. With 100 and 150 profiling traces, the benefit of the shorter traces is only observed at $n_{\text{attack}} = 1$ or 2, and beyond that, longer traces consistently yield better performance. This change in behavior can be explained, in part, by examining the numerical stability of the covariance matrices used in the template attack.

More specifically, the reciprocal condition number (rcond) of the covariance matrix used during the modeling of each class differs significantly between the two trace sets. For the 1140-sample traces, the rcond value is extremely low (~ 0.000001), indicating that the covariance matrix is poorly conditioned and nearly singular. This leads to numerical instability during the inversion required in LDA and template computations, making the model highly sensitive to noise and estimation errors. In contrast, the covariance matrix computed from the 114-sample traces has a much higher rcond value (~ 0.6), suggesting it is well-conditioned and numerically stable. This stability leads to more reliable template estimation, which explains why, in low-trace regimes (few profiling and attack traces), the shorter traces can outperform the longer ones—even though they carry less raw information.

5.3 Complexity and performance analysis

Above we have seen that several factors such as the number of leakage samples or the number of traces affect the results of our attacks considerably. Furthermore, in our evaluation we have used the lowest security parameters of Classic McEliece ($m = 12$), for simplicity of the experiments. Hence, we provide here a complexity analysis of the two main steps (profiling and attack) of our attack, so that we can estimate the cost of the attacks for other parameters.

As shown in Section 3.5, during the profiling step we compute 2^m parameters (mean vectors and covariance matrices). The highest complexity of these parameters is given by the computation of the covariance matrices, which requires $O(n_p \cdot n_s^2)$ operations. Hence, the profiling step requires $O(2^m \cdot n_p \cdot n_s^2)$ operations in total.

For the attack step, we first need to note the use of LDA as a compression step and then the use of the linear discriminant for the actual classification part. LDA requires the computation of the covariance matrices, which was already done during profiling and hence we do not count it for the complexity estimation of the attack. However, LDA reduces the size of the attack traces from n_s to some $\hat{n}_s \ll n_s$ (e.g. in some of our experiments from 1140 to 10). LDA also reduces the size of the mean vectors and covariance matrices, which are projected to match the same dimension $\hat{n}_s$. Furthermore, note that in the linear discriminant (Eq. 10), the first and last terms ($\bar{\mathbf{x}}'_\alpha \mathbf{S}_{\text{pooled}}^{-1}$, respectively $\bar{\mathbf{x}}'_\alpha \mathbf{S}_{\text{pooled}}^{-1} \bar{\mathbf{x}}_\alpha$) can be precomputed in time $O(\hat{n}_s^2)$, which is clearly lower than the time required for the computation of the covariance matrices ($O(n_p \cdot n_s^2)$) and hence we can include these

precomputations as part of the profiling step. We are left with the summation of the n_a vectors, which is done in $O(n_a \cdot \hat{n}_s)$. Hence, the total attack time per α equals $O(2^m \cdot n_a \cdot \hat{n}_s)$. This implies that the overall attack, for completely retrieving the secret Goppa support is $O(2^{2m} \cdot n_a \cdot \hat{n}_s)$.

Table 6: Timings (in seconds) for our attack for LDA dimension equal to 10 and $m = 12$.

Samples	Profiling	Attack ($n_{attack} = 1$)	Attack ($n_{attack} = 10$)
1140	3058	7	68
114	128	6.5	65

Performance timings: We implemented our attack using Matlab - 9.2.0.538062 (R2017a), with pre-compilation for execution on Intel machines. We have performed timing estimations for both the profiling as well as for the attack phase on a NVIDIA DGX A100 System equipped with two AMD EPYC 7742 CPUs and 2 TB of memory. We show the timings in Table 6. For the profiling step, we can see a great advantage when going from $n_s = 1140$ to $n_s = 114$, as the timing decreases by almost two orders of magnitude, which is as expected from the complexity estimations above (note the timing is directly proportional to n_s^2). As for the attack step, when using $n_{attack} = 1$ attack trace, the attack only requires up to 7 seconds on average², while for $n_{attack} = 10$ the attack step requires below 70 seconds, even when using $n_s = 1140$, for which the guessing entropy is zero. Note that in all cases, the timings show the attack time to compute the average guessing entropy across all the elements of the Goppa support. In terms of computation time, this is roughly equivalent to the time required to recover the entire secret Goppa support $\mathcal{L}$.

Finally, based on our complexity estimates, we can see that going from $m = 12$ to $m = 13$ would only involve a multiplicative factor of 4 in the attack step ($2^{2m} = 2^{24}$ vs $2^{2m} = 2^{26}$), leading to an estimated attack time of roughly 30 seconds when using $m = 13$ for $n_s = 1140$.

6 Possible countermeasures and attack extensions

We shall first focus on possible countermeasures against our attack. Recall that any plain-text $\mathbf{e}$ is encrypted into $\mathbf{z} = \mathbf{H}_{\text{pub}} \mathbf{e}^T$ which is afterwards taken as input by the Decapsulation algorithm. Hence, all side-channel attacks that fall under the weight one plain-text model, and implicitly our attack, can be avoided using a check function. Indeed, the Decapsulation algorithm could begin with a for loop where we check whether $\mathbf{z} = \mathbf{H}_{\text{pub}}[i]$ for $i \in \llbracket 0; n-1 \rrbracket$. If comparing equality of two mt -length binary vectors can be done in constant time, and since there are n columns, the time complexity of this verification is $O(n)$, when $n \rightarrow \infty$. Overall, this checkup would not significantly increase the execution time of the Decapsulation.

Since the aforementioned countermeasure can be deployed before the application of the Berlekamp-Massey algorithm, then our attack would fail as well. However, as we shall demonstrate in the next paragraph, the Berlekamp-Massey leaks information even for higher weights models. We will detail the case $\text{wt}(\mathbf{e}) = 2$.

Suppose we have a plain-text that satisfies $\text{wt}(\mathbf{e}) = 2$, to be more precise, $e_i = e_j = 1$ and $e_l = 0, \forall l \neq i, l \neq j$. This implies that $\mathbf{s} = (g(\alpha_i)\alpha_i^l + g(\alpha_j)\alpha_j^l)_{l=\llbracket 0; 2t-1 \rrbracket}$. After the fourth iteration of the Berlekamp-Massey algorithm applied to $\mathbf{s}$ we have that $C = [1, \alpha_i + \alpha_j, \alpha_i\alpha_j]$. This implies that for any of the remaining steps the discrepancy will

²We have obtained similar attack timings also on a MacBookPro 2014 Computer.

always be equal to 0. Indeed,

$$\begin{aligned}
d &= s_{l+1} - s_l \mathbf{C}_1 - s_{l-1} \mathbf{C}_2 \\
&= g(\alpha_i) \alpha_i^{l+1} + g(\alpha_j) \alpha_j^{l+1} - (g(\alpha_i) \alpha_i^l + g(\alpha_j) \alpha_j^l) (\alpha_i + \alpha_j) \\
&\quad - (g(\alpha_i) \alpha_i^{l-1} + g(\alpha_j) \alpha_j^{l-1}) (\alpha_i \alpha_j) \\
&= 0.
\end{aligned}$$

Notice that at each step the values C_1, C_2 are multiplied with different elements in $\mathbb{F}_{2^m}$ (the syndrome components). Such a multiplication is repeated $2t - 4$ times, enough to discriminate the elements $C_1 = \alpha_i + \alpha_j$ and $C_2 = \alpha_i \alpha_j$. In addition, there is a one-to-one mapping between the polynomial $\sigma(x) = x^2 + (\alpha_i + \alpha_j)x + \alpha_i \alpha_j$ and its roots (α_i, α_j) .

To test our hypothesis we have performed a TVLA similar to the weight one case, by modifying Algorithm 5. We have two fixed values $\alpha_{f_1}, \alpha_{f_2}$ and four possible cases

- **Both values fixed:** $\mathcal{L}_K(i) = \alpha_{f_1}, \mathcal{L}_K(j) = \alpha_{f_2}$ which leads to computing the set $\mathcal{T}_{f_1, f_2}$ of traces from BM when applied to $\mathbf{z} = \mathbf{H}_{\text{pub}} \mathbf{e}$ and $e_i = e_j = 1$.
- **First value fixed, second random:** $\mathcal{L}_K(i) = \alpha_{f_1}, \mathcal{L}_K(j) \neq \alpha_{f_2}$ which leads to computing the set $\mathcal{T}_{f_1, r_2}$. Notice that when α_{f_1} is fixed, for any $\alpha_{r_2} \neq \alpha_{f_2}$, as long as $\alpha_{f_1} \neq 0$ we have

$$\alpha_{f_1} + \alpha_{f_2} \neq \alpha_{f_1} + \alpha_{r_2} \text{ and } \alpha_{f_1} \alpha_{f_2} \neq \alpha_{f_1} \alpha_{r_2}.$$

- **First value random, second fixed:** $\mathcal{L}_K(i) \neq \alpha_{f_1}, \mathcal{L}_K(j) = \alpha_{f_2}$ which leads to computing the set $\mathcal{T}_{r_1, f_2}$. Here, when α_{f_2} is fixed, for any $\alpha_{r_1} \neq \alpha_{f_1}$, as long as $\alpha_{f_2} \neq 0$ we have

$$\alpha_{f_1} + \alpha_{f_2} \neq \alpha_{r_1} + \alpha_{f_2} \text{ and } \alpha_{f_1} \alpha_{f_2} \neq \alpha_{r_1} \alpha_{f_2}.$$

- **Both values random:** $\mathcal{L}_K(i) \neq \alpha_{f_1}, \mathcal{L}_K(j) \neq \alpha_{f_2}$ which leads to computing the set $\mathcal{T}_{r_1, r_2}$. In this case, we can get collisions

$$\alpha_{f_1} + \alpha_{f_2} = \alpha_{r_1} + \alpha_{r_2} \text{ or } \alpha_{f_1} \alpha_{f_2} = \alpha_{r_1} \alpha_{r_2}.$$

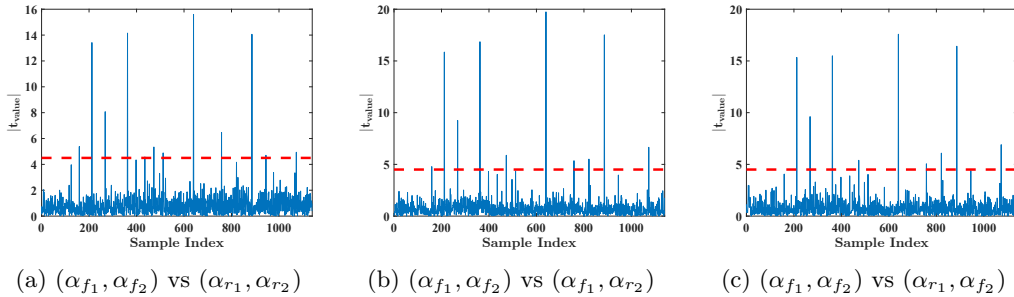


Figure 5: Point-wise TVLA for weight 2 plain-text model on the BM for MCELTCE348864.

In Figure 5 we illustrate our TVLA experiments. All three sub-figures show high peaks even for the worst case, namely when the two values are random $\alpha_{r_1}, \alpha_{r_2}$ and where either the sum of the product could be equal to the fixed values $\alpha_{f_1} + \alpha_{f_2}, \alpha_{f_1} \alpha_{f_2}$. This indicates that our method has a high potential of applicability for higher Hamming weights.

One may use our Template Attack strategy to exploit the observed leakage: firstly build profiles for the sum $(\alpha_i + \alpha_j)$ and product $(\alpha_i \cdot \alpha_j)$ separately. Then, for a given trace, find the most likely sums and the most likely products. Afterwards, try to infer the most probable values for α_i and α_j , in decreasing order of likelihood (trying the most likely combinations). Such approach only requires profiles of the same size as for the experiments for $\text{wt}(\mathbf{e}) = 1$ (2^m parameters for the sum and 2^m parameters for the product).

7 Conclusions

The reference implementation of the *Classic McEliece* KEM was subject to various side-channel attacks, targeting almost all functions in the decapsulation process, except for the support generation and Berlekamp-Massey decoding algorithm. Here, we provide a first side-channel attack targeting the BM algorithm, crafted on the chosen plaintext model and using small Hamming weight plaintexts.

We have performed efficient template attacks against the reference implementation of the MCELIECE348864 parameter set, running on the ARM Cortex M4 using the Chip Whisperer platform. Using a single attack trace per secret coefficient, we can perfectly recover the entire Goppa secret support.

The vulnerability that we identify in the BM is unavoidable in the current implementation, and no practical means of mitigating it exist as of now without incurring undesirable overhead. Possible paths towards a side-channel resistant implementation include investigation of other decoders, such as the Patterson decoding algorithm or the Interpolation method by Bernstein, as well as implementation variants of the BM.

Another step in the decapsulation process, which has not been analyzed up-to-date, is the SUPPORT_GEN. A thorough investigation from a side-channel perspective of this step is among the future task we aim to accomplish.

References

- [AAB⁺22] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [ABC⁺22] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- [APSQ06] Cédric Archambeau, Eric Peeters, F X Standaert, and J J Quisquater. Template attacks in principal subspaces. In *Cryptographic Hardware and Embedded Systems-CHES 2006: 8th International Workshop, Yokohama, Japan, October 10-13, 2006. Proceedings 8*, pages 1–14. Springer, 2006.
- [BCM⁺25] Marcus Brinkmann, Chitchanok Chuengsatiansup, Alexander May, Julian Nowakowski, and Yuval Yarom. Leaky mceliece: Secret key recovery from highly erroneous side-channel information. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):94–125, Mar. 2025.
- [BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*, pages 16–29. Springer, 2004.
- [BDK⁺25] Sebastian Bitzer, Jeroen Delvaux, Elena Kirshanova, Sebastian Maaßen, Alexander May, and Antonia Wachter-Zeh. How to lose some weight: a prac-

- tical template syndrome decoding attack. *Designs, Codes and Cryptography*, 2025.
- [Ber15] Elwyn R. Berlekamp. *Algebraic Coding Theory - Revised Edition*. World Scientific Publishing Co., Inc., USA, 2015.
- [Ber24] Daniel J. Bernstein. Understanding binary-Goppa decoding. *IACR Communications in Cryptology*, 1(1), 2024.
- [BMvT78] Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Trans. Inf. Theory*, 24(3):384–386, 1978.
- [CC21] Ming-Shing Chen and Tung Chou. Classic McEliece on the ARM Cortex-M4. *IACR TCHES*, 2021(3):125–148, 2021.
- [CCD⁺21] Pierre-Louis Cayrel, Brice Colombier, Vlad-Florin Dragoi, Alexandre Menu, and Lilian Bossuet. Message-recovery laser fault injection attack on the Classic McEliece cryptosystem. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 438–467. Springer, 2021.
- [CDCG22] Brice Colombier, Vlad-Florin Dragoi, Pierre-Louis Cayrel, and Vincent Grosso. Profiled side-channel attack on cryptosystems based on the binary syndrome decoding problem. *IEEE Trans. Inf. Forensics Secur.*, 17:3407–3420, 2022.
- [CGCD25] Brice Colombier, Vincent Grosso, Pierre-Louis Cayrel, and Vlad Dragoi. Message-recovery Horizontal Correlation Attack on Classic McEliece. In *Conference on Constructive Approaches for Security Analysis and Design of Embedded Systems (CASCADE 2025)*, Saint-Étienne, France, April 2025.
- [CK14] Marios O Choudary and Markus G Kuhn. Efficient stochastic methods: Profiled attacks beyond 8 bits. In *International Conference on Smart Card Research and Advanced Applications*, pages 85–103. Springer, 2014.
- [CK17] Marios O Choudary and Markus G Kuhn. Efficient, portable template attacks. *IEEE Transactions on Information Forensics and Security*, 13(2):490–501, 2017.
- [CRR02] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template attacks. In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*, pages 13–28. Springer, 2002.
- [DCCR13] Vlad Dragoi, Pierre-Louis Cayrel, Brice Colombier, and Tania Richmond. Polynomial structures in code-based cryptography. In Goutam Paul and Serge Vaudenay, editors, *Progress in Cryptology – INDOCRYPT 2013*, pages 286–296, Cham, 2013. Springer International Publishing.

- [DCV⁺24] Vlad-Florin Drăgoi, Brice Colombier, Nicolas Vallet, Pierre-Louis Cayrel, and Vincent Grosso. Full key-recovery cubic-time template attack on classic mceliece decapsulation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):367–391, Dec. 2024.
- [DRBL18] Vlad Dragoi, Tania Richmond, Dominic Bucerzan, and Axel Legay. Survey on cryptanalysis of code-based cryptography: From theoretical to physical attacks. In *2018 7th International Conference on Computers Communications and Control (ICCCC)*, pages 215–223, 2018.
- [fSidI25] Bundesamt für Sicherheit in der Informationstechnik. Bsi tr-02102-1. <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html>, Jan 2025. Standard.
- [GCCD23] Vincent Grosso, Pierre-Louis Cayrel, Brice Colombier, and Vlad-Florin Dragoi. Punctured syndrome decoding problem - efficient side-channel attacks against Classic McEliece. In Elif Bilge Kavun and Michael Pehl, editors, *Constructive Side-Channel Analysis and Secure Design - 14th International Workshop, COSADE 2023, Munich, Germany, April 3-4, 2023, Proceedings*, volume 13979 of *Lecture Notes in Computer Science*, pages 170–192. Springer, 2023.
- [GGJR⁺11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.
- [GJJ22a] Qian Guo, Andreas Johansson, and Thomas Johansson. A key-recovery side-channel attack on classic McEliece implementations. *IACR TCES*, 2022(4):800–827, 2022.
- [GJJ22b] Qian Guo, Andreas Johansson, and Thomas Johansson. A key-recovery side-channel attack on Classic McEliece implementations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):800–827, 2022.
- [GMO01] Karine Gandolfi, Christophe Mourtél, and Francis Olivier. Electromagnetic analysis: Concrete results. In *Cryptographic Hardware and Embedded Systems—CHES 2001: Third International Workshop Paris, France, May 14–16, 2001 Proceedings 3*, pages 251–261. Springer, 2001.
- [Gop70] Valerii Denisovich Goppa. A new class of linear correcting codes. *Problemy Peredachi Informatsii*, 6(3):24–30, 1970.
- [Gro96] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [HGDM⁺11] Gabriel Hospodar, Benedikt Gierlichs, Elke De Mulder, Ingrid Verbauwhede, and Joos Vandewalle. Machine learning in side-channel analysis: a first study. *Journal of Cryptographic Engineering*, 1(4):293–302, 2011.
- [HGG20] Benjamin Hettwer, Stefan Gehrér, and Tim Güneysu. Applications of machine learning techniques in side-channel attacks: a survey. *Journal of Cryptographic Engineering*, 10(2):135–162, 2020.
- [Jol02] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics. Springer-Verlag, New York, 2002.

-
- [JW⁺02] Richard Arnold Johnson, Dean W Wichern, et al. *Applied multivariate statistical analysis*. Prentice hall Upper Saddle River, NJ, 2002.
 - [KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 388–397. Springer, 1999.
 - [KM23] Elena Kirshanova and Alexander May. Breaking Goppa-based McEliece with hints. *Inf. Comput.*, 293:105045, 2023.
 - [LNPS20] Norman Lahr, Ruben Niederhagen, Richard Petri, and Simona Samardjiska. Side channel information set decoding using iterative chunking - plaintext recovery from the "Classic McEliece" hardware reference implementation. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 881–910. Springer, 2020.
 - [LS01] Pierre Loidreau and Nicolas Sendrier. Weak keys in the mceliece public-key cryptosystem. *IEEE Transactions on Information Theory*, 47(3):1207–1211, 2001.
 - [Mas69] James L. Massey. Shift-register synthesis and BCH decoding. *IEEE Trans. Inf. Theory*, 15:122–127, 1969.
 - [McE78] Robert J McEliece. A public-key cryptosystem based on algebraic. *Coding Thv*, 4244:114–116, 1978.
 - [MNB⁺25] Hannah J. Manetsch, Gyohei Nomura, Elie Bataille, Xudong Lv, Kon H. Leung, and Manuel Endres. A tweezer array with 6100 highly coherent atomic qubits. *Nature*, 2025.
 - [MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error correcting codes*, volume 16. Elsevier, 1977.
 - [oST24a] National Institute of Standards and Technology. Fips 203 module-lattice-based key-encapsulation mechanism standard. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>, Aug 2024. Standard.
 - [oST24b] National Institute of Standards and Technology. Fips204 module-lattice-based digital signature standard. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>, Aug 2024. Standard.
 - [oST24c] National Institute of Standards and Technology. Fips205 stateless hash-based digital signature standard. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>, Aug 2024. Standard.
 - [Pat75] N. Patterson. The algebraic decoding of Goppa codes. *IEEE Transactions on Information Theory*, 21(2):203–207, 1975.
 - [PFH⁺22] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.

- [PGD⁺22] Sabine Pircher, Johannes Geier, Julian Danner, Daniel Mueller-Gritschneider, and Antonia Wachter-Zeh. Key-recovery fault injection attack on the Classic McEliece KEM. In Jean-Christophe Deneuville, editor, *Code-Based Cryptography - 10th International Workshop, CBCrypto 2022, Trondheim, Norway, May 29-30, 2022, Revised Selected Papers*, volume 13839 of *Lecture Notes in Computer Science*, pages 37–61. Springer, 2022.
- [PPM⁺23] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. Sok: Deep learning-based physical side-channel analysis. *ACM Computing Surveys*, 55(11):1–35, 2023.
- [QS01] Jean-Jacques Quisquater and David Samyde. Electromagnetic analysis (ema): Measures and counter-measures for smart cards. In *Smart Card Programming and Security: International Conference on Research in Smart Cards, E-smart 2001 Cannes, France, September 19–21, 2001 Proceedings*, pages 200–210. Springer, 2001.
- [RPC22] Anca Rădulescu, Pantelimon G Popescu, and Marios O Choudary. Ge vs gm: Efficient side-channel security evaluations on full cryptographic keys. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 886–905, 2022.
- [SA08] François-Xavier Standaert and Cédric Archambeau. Using subspace-based template attacks to compare and combine power and electromagnetic information leakages. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 411–425. Springer, 2008.
- [SCD⁺23] Boly Seck, Pierre-Louis Cayrel, Vlad-Florin Drăgoi, Idy Diop, Morgan Barbier, Jean Belo Klamti, Vincent Grosso, and Brice Colombier. A side-channel attack against classic McEliece when loading the goppa polynomial. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 105–125. Springer, Cham, July 2023.
- [Sen00] N. Sendrier. Finding the permutation between equivalent linear codes: the support splitting algorithm. *IEEE Transactions on Information Theory*, 46(4):1193–1203, 2000.
- [Sho94] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [SMY09] François-Xavier Standaert, Tal G Malkin, and Moti Yung. A unified framework for the analysis of side-channel key recovery attacks. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 443–461. Springer, 2009.
- [VCC⁺25] Nicolas Vallet, Pierre-Louis Cayrel, Brice Colombier, Vlad-Florin Drăgoi, and Vincent Grosso. Optimizing key recovery in classic mceliece: Advanced error correction for noisy side-channel measurements. *IACR Communications in Cryptology*, 2(2), 2025.
- [VCMKS12] Nicolas Veyrat-Charvillon, Marcel Medwed, Stéphanie Kerckhof, and François-Xavier Standaert. Shuffling against side-channel attacks: A comprehensive study with cautionary note. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology – ASIACRYPT 2012*, pages 740–757, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.